Unity PSG Player

This is a library for synthesis PSG (Programmable Sound Generator) sound sources such as retro game consoles on Unity. The performance data is MML (Music Macro Language) text, which describes musical notation in text, allowing easy creation of music data. The library is designed to produce expressions similar to those of NES sound generators (except DPCM).

- The sound generator can produce four types of square waves, a triangle wave, and two types of noise. The triangle wave is a 4-bit waveform.
- Performance expressions include sweep, LFO (vibrato), and volume envelope.

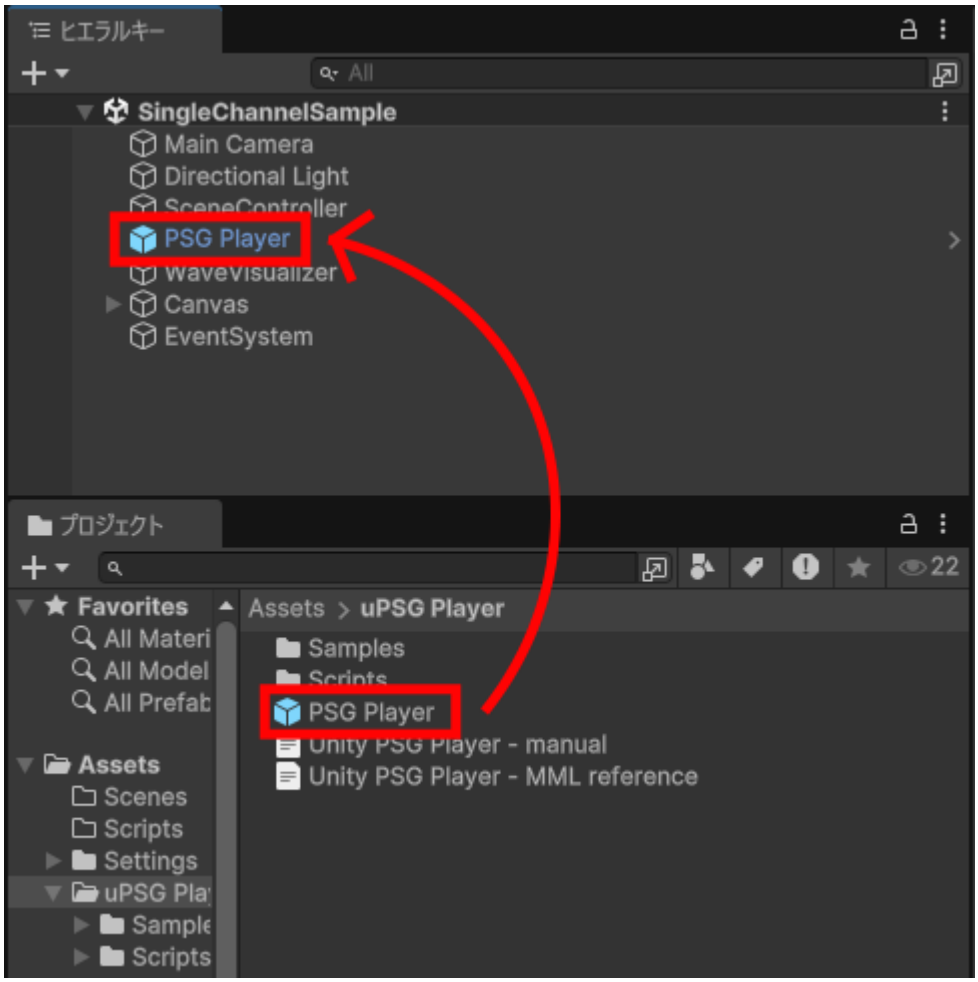
> I'm not aiming for a perfect recreation of the Famicom sound source. (It's too much trouble.)
> My goal is solely to produce sounds using only Unity, without preparing separate sound clips.

The PSG Player produces monophonic (single-note) sound. To play NES 4-note polyphonic sounds (excluding DPCM), you can achieve this by preparing multiple PSG Players and playing them simultaneously.

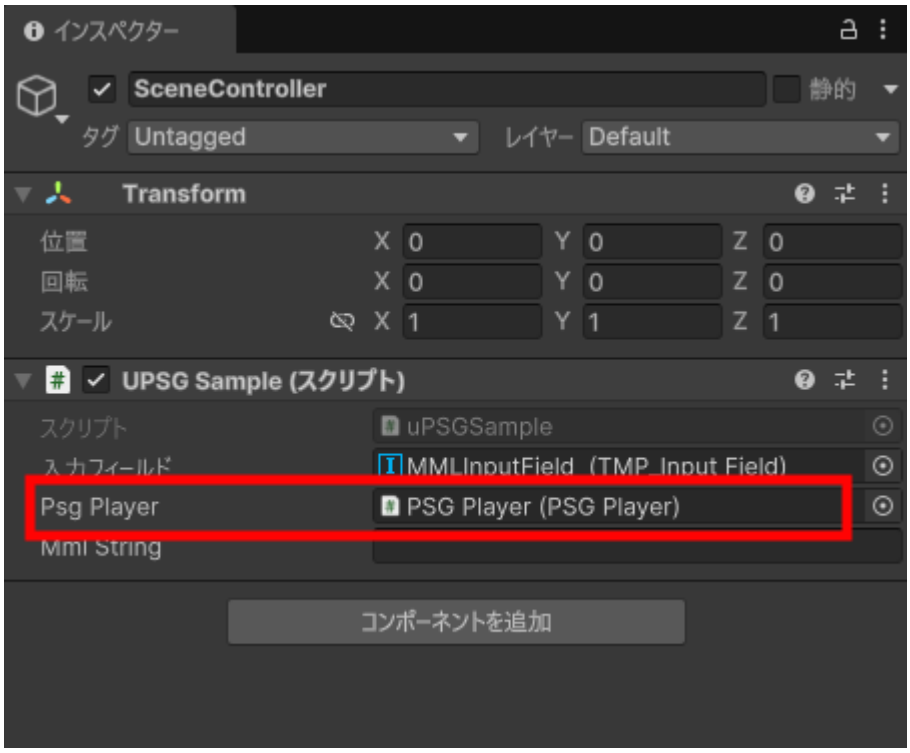
Quick Guide

Basic Usage

1. Place the PSG Player prefab in the Hierarchy.



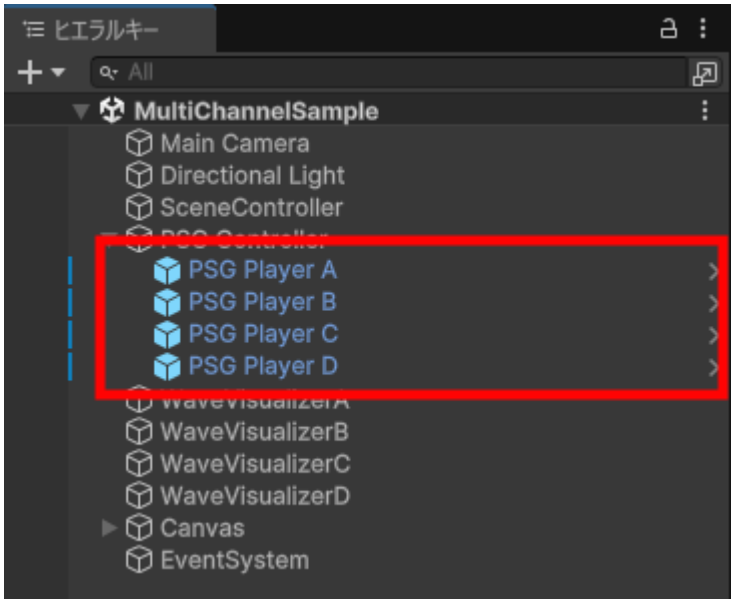
2. Prepare a PSGPlayer class variable in the script you are operating, and attach the PSG Player object you have placed.



3. The MML written in the mmlString variable of the PSGPlayer will be played by Play() function. For details on MML, refer to the MML Reference.

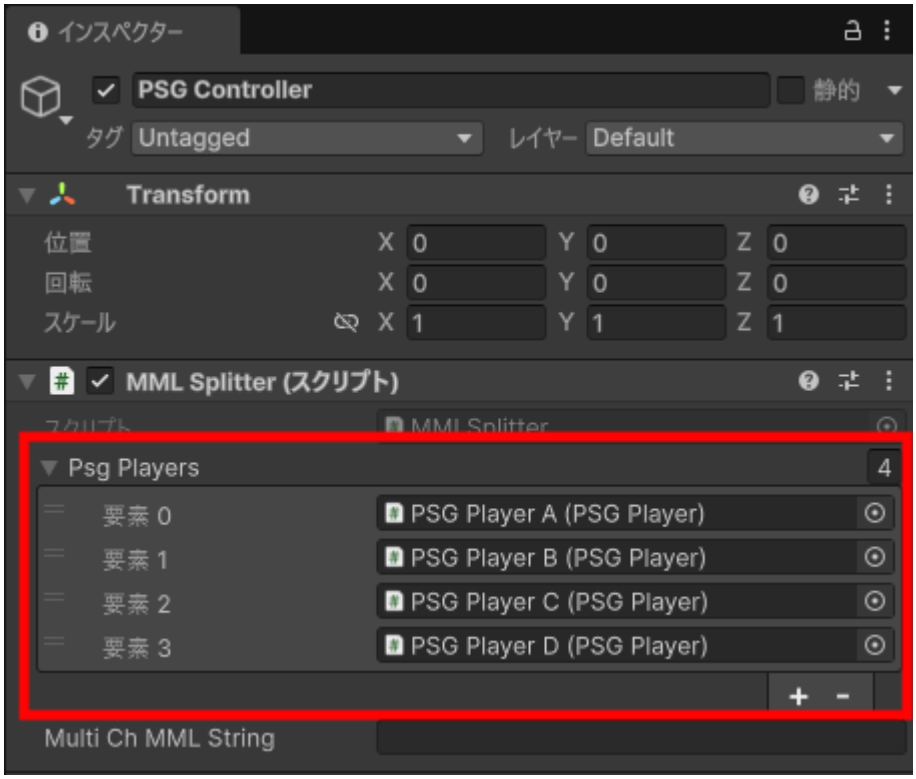
Multi-channel Usage

1. Place the PSG Player prefab according to the required number of channels.

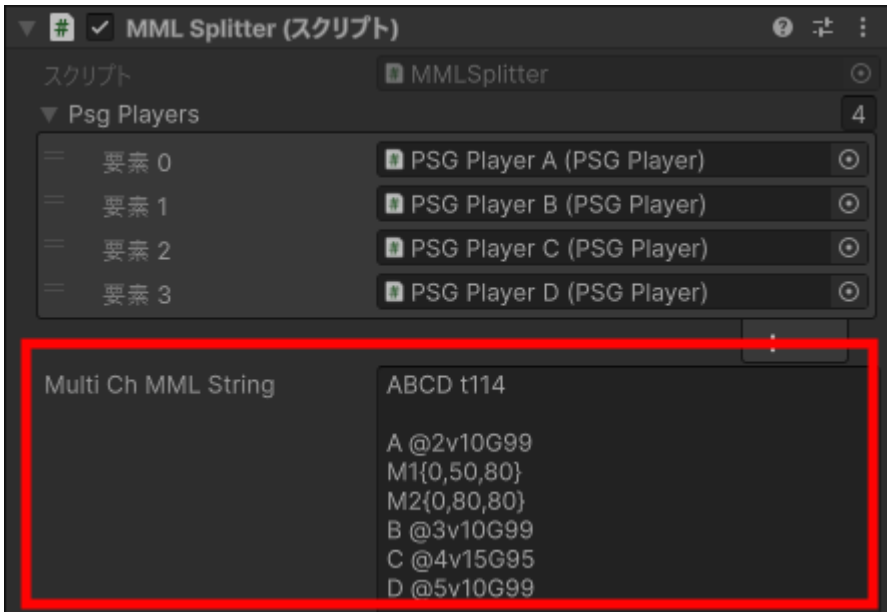


2. Attach the MMLSplitter script to the appropriate game object.

3. Assign the placed PSG Player to the psgPlayers of the MMLSplitter from the Inspector.



4. Place the MML into the multiChMMLString variable of MMLSplitter, distribute the MML to each channel using the SplitMML() function, and play it using the PlayAllChannels() function.



About MML

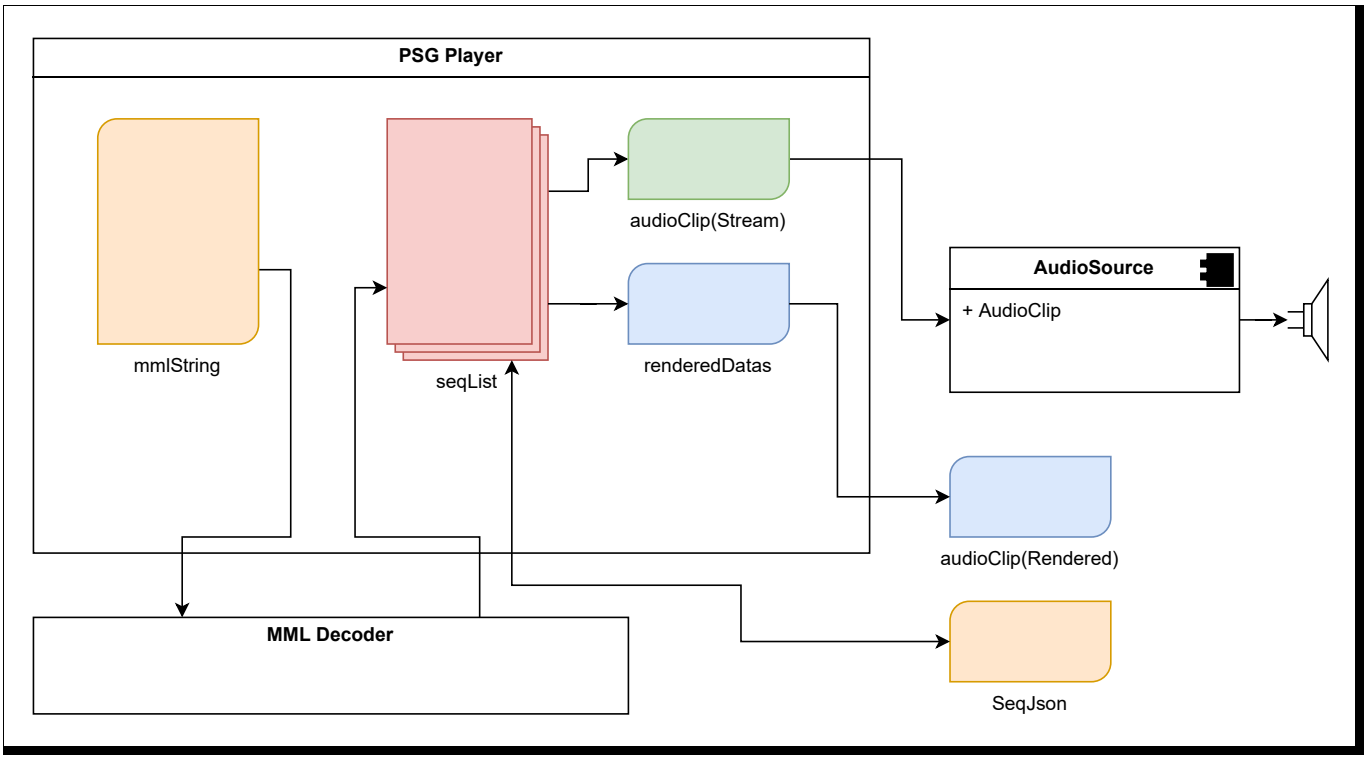
MML (Music Macro Language) is a text-based data description language that uses letters and numbers (e.g., cdefgab) to describe notes, note durations, octaves, and other musical elements, enabling computers to automatically play music.
For information on the MML supported by the Unity PSG Player, please refer to "MML Reference".

Composition

The Unity PSG Player consists of:

- The "**PSG Player**" that synthesizes sound according to sequence data
- The "**MML Decoder**" that converts MML to sequence data
- The "**MML Splitter**" that splits MML

MML Splitter is not required for single sound use.
An AudioSource component is also required to play the sound generated by PSG Player.

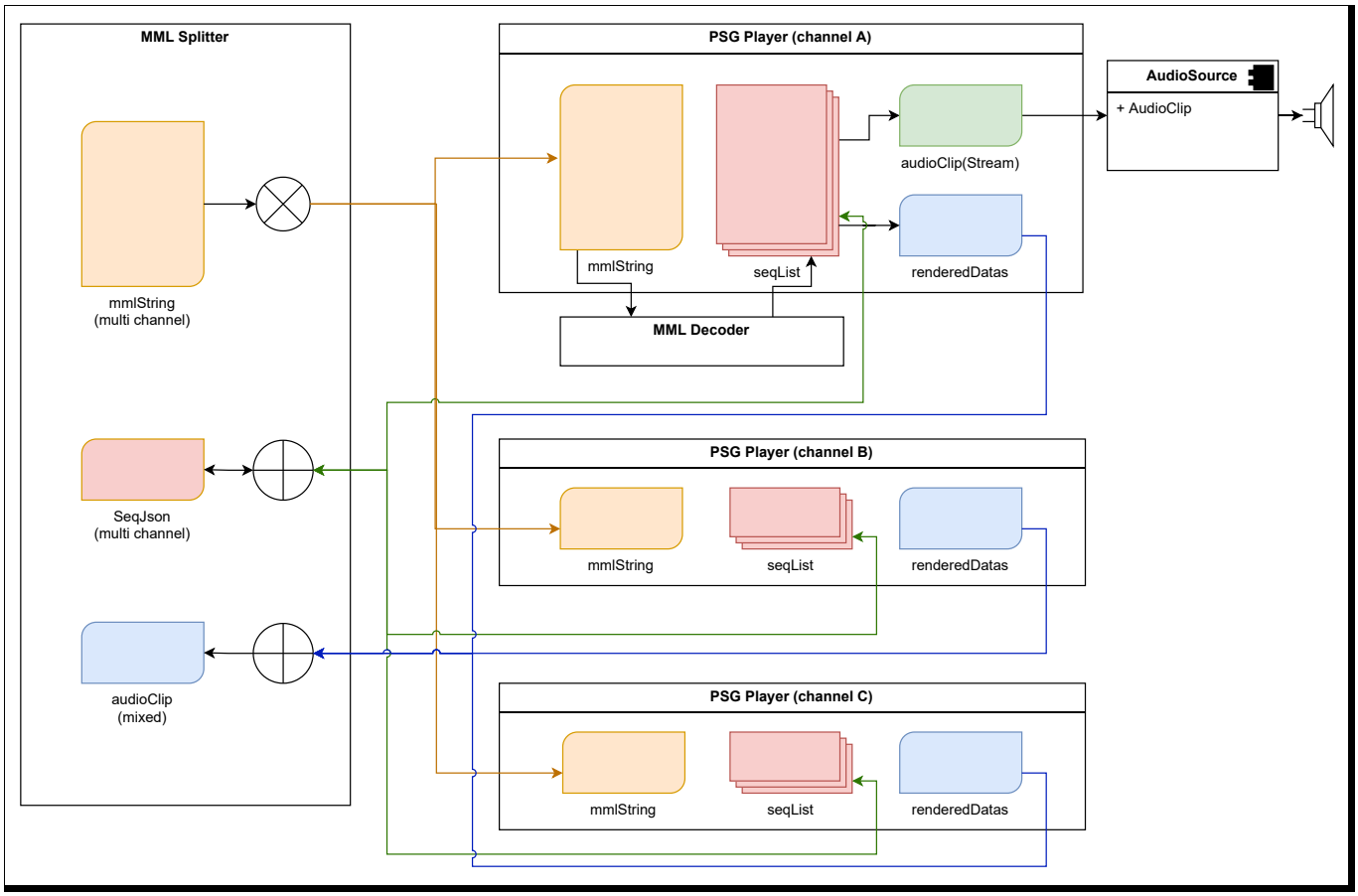


PSG Player stores MML—which represents musical notation as a string—in the mmlString variable, but to actually generate sound, it must be converted into sequence data in a List array, the seqList variable.

Conversion from MML to sequence data is performed by the **MML Decoder**.
The MML Decoder parses MML one character at a time, converts it into event data consisting of “Command, Parameter, Length,” and returns it as a sequence of data.
During playback or rendering, the sequence data is processed sequentially to calculate pitch and volume, and square waves, triangle waves, and noise are generated as monophonic sounds.

There are essentially two methods for generating sound: “**Stream playback**” and “**Rendering**.”
While an AudioClip is being streamed, it sequentially requests the necessary amount of data for the buffer, and the PSG Player calculates the waveform data each time.
Rendering calculates the entire sequence frame by frame to generate waveform data.
The rendered data can be treated as a standard AudioClip.

Sequence data can be exported and imported in JSON format.



To split multi-channel MML, use the **MML Splitter**.
It processes each line of the MML and determines the destination channel based on the characters at the beginning of the line. For details, refer to the MML Reference. Besides distributing MML, the MML Splitter can also centralize control over each PSG Player.
You can also mix the waveform data generated by each PSG Player and output it as a single AudioClip.

When playing sound effects using PSG, retro game consoles would interrupt the playback of one BGM channel to play sound effects in real time. However, the PSG Player buffers pre-loaded sounds, making it impossible to play sound effects with precise timing.
To play sound effects in PSG Player similarly to retro game consoles, prepare a separate PSG Player instance dedicated solely to sound effects and mute one of the BGM channels.

When using PSG Player in multi-channel mode, using the AudioMixer makes it easier to adjust volume levels and other settings.

About Sample rate

PSG Player allows you to set the sample rate for generated AudioClips. The sample rate is the number of samples per second. Since processing occurs for each sample, increasing the sample rate also increases CPU load.

Additionally, the sample rate affects the upper limit of the frequency of the generated sound.
The number of samples required per wavelength varies depending on the tone, as follows:

Tone	Samples
Square wave	2 samples

Tone	Samples
25%(75%) Pulse wave	4 samples
12.5% Pulse wave	8 samples
Triangle wave	32 samples

The frequency of sound that can be produced without issues is determined by dividing the sample rate by the number of samples mentioned above, ~~so the highest pitch achievable, especially for a triangle wave, will be lower~~.

`v0.9.2beta` Slightly modified the triangle wave generation logic, enabling it to produce high frequencies comparable to those of a square wave.

- This represents the upper limit for achieving the expected pitch; higher notes will cause the tone to deteriorate.
  At the default sampling rate, proper sound reproduction is only guaranteed up to approximately the seventh octave (o7).
- Due to unique noise processing, lower sample rates will cause volume to decrease at high frequencies.

Musical Expression

PSG Player allows you to use volume envelopes, sweeps, and LFOs for performance expression.
However, noise sounds and sounds generated by frequency specification will disable sweep and LFO effects that alter pitch.

About Rendering

`v0.9.6beta`

PSG Player has been developed assuming the use of the Stream from AudioClip.Create().
Stream playback distributes the load, so even long sequences like background music can be played with reasonably good responsiveness.
However, since buffering occurs when Create() is called, there was some concern about playing timing-sensitive sound effects.

Rendering outputs the entire sequence as waveform data for an AudioClip.
By preparing this waveform data in advance, you can achieve performance equivalent to playing a standard audio file.
However, rendering is computationally intensive, so converting long time performances may cause the program to freeze.

Using asynchronous rendering allows you to return to the main thread at regular intervals, thereby distributing the load.
Instead, the time required to complete rendering will increase.
We recommend using **streams**, **rendering**, and **asynchronous rendering** appropriately depending on the use case.

Note that during rendering, the sequence loop command (MML loop “L”) is disabled.
(There's this mysterious Unity behavior where loop points embedded in audio files get applied to AudioClips,

but there's no way to add them when creating them manually...)

By the way, in WebGL, the Stream from AudioClip.Create() is unsupported (it plays but the buffer doesn't update), but rendered AudioClips can be played.

About Sequence Data

The sequence data seqList of the PSG Player consists of a List<T> of the SeqEvent class.
During playback or rendering, this List is processed sequentially, and the index is adjusted backward during loops and repeats.

The SeqEvent class has member variables for the command (seqCmd), parameters (seqParam), and step size (seqStep).
The commands are defined as constants in the SEQ_CMD enumeration.
Parameters specify the values required to execute a command.
Even for commands that don't require parameters, enter some value (usually 0).

The step size specifies the time remaining until the next event in ticks.
The length of a single tick is calculated based on the resolution (tick/note) specified in tickPerNote and the tempo (note/60sec).
Except for notes (NOTE_ON) and rests (REST), the duration is set to 0.

Sequence data can be exchanged with external systems using GetSeqJson() and SetSeqJson(), along with tickPerNote.
Additionally, serialization allows you to export and import JSON-formatted strings using ExportSeqJson() and ImportSeqJson().

Sequence commands

SEQ_CMD	seqParam
PROGRAM_CHANGE	Tone number
SET_TEMPO	Tempo specification
TUNE	o4a frequency
NOTE_ON	Note Number[^1]
REST	0
NOTE_TIE	0
GATE_STEP_RATE	Ratio of gate lengths
DIRECT_FREQ	Frequency
VOLUME	Volume
ENV_ON	Envelope ID
ENV_PARAM_START	Envelope ID (Off when set to 0)
ENV_PARAM	Volume (-1 for loop point)

SEQ_CMD	seqParam
ENV_PARAM_END	0
SWEEP	Sweep change
LFO_SET	LFO ID (Off when set to 0)
LFO_DELAY	LFO Delay
LFO_SPEED	LFO Speed
LFO_DEPTH	LFO Depth
LFO_PARAM_END	0
LOOP_POINT	0
REPEAT_START	0
REPEAT_END	Number of repeats
END_OF_SEQ	0

[^1]: If the note number is a negative value (note number * -1), set the gate to 100% to treat the note as part of a tied note.